

Job Portal Backend

A modular, domain-driven REST API for a two-sided hiring marketplace

Stack: NestJS 11 · TypeScript · PostgreSQL / TypeORM · Supabase Auth (JWKS) · Docker (multi-stage) · OpenAPI/Swagger

Prepared: 2 September 2026

9 NestJS modules	24 REST endpoints	7 TypeORM migrations	296 passing E2E tests	96.2% statement coverage
----------------------------	-----------------------------	--------------------------------	---------------------------------	------------------------------------

Structural metrics counted directly from the source tree; test/coverage figures from a live run of the full E2E suite against PostgreSQL in Docker (see Appendix A).

~3,500 req/sec avg (GET /jobs, load test)	17 ms p97.5 latency	0 failed requests (70k+ served)
---	-------------------------------	---

Load test: autocannon, 50 connections, 20 s, against GET /jobs on a single local instance with the request-rate limiter raised for the test — see the caveat in §5 and method in Appendix C.

Contents

Contents.....	2
1. Overview	3
2. System Design & Architecture Highlight	3
3. Request Lifecycle	4
4. Technology Stack	4
5. Measurable Results	4
Appendix A — Reproducing the coverage numbers.....	5
Appendix B — Full Coverage Report (per file)	6
Appendix C — Producing a real throughput number	9

1. Overview

Job Portal Backend is the server for a two-sided hiring platform: job seekers build profiles and apply to roles; employers publish jobs and review applicants. It is built as a modular NestJS monolith with clear domain boundaries, a Repository-based persistence layer, and a Strategy-based storage abstraction, and it delegates identity entirely to Supabase so the API itself stays stateless.

Design principle — the API issues no tokens of its own. Authentication is offloaded to Supabase and verified locally against its JWKS, which keeps the service horizontally scalable (no session store) and removes credential handling from the application entirely.

2. System Design & Architecture Highlight

2.1 Modular domain decomposition

Each business domain is an isolated NestJS module with its own controllers, DTOs, entity, and repository service. Modules communicate through injected services and an in-process event bus rather than reaching into each other's internals.

- **Jobs** — public + authored job CRUD, lifecycle status transitions, paginated public browsing.
- **Applications** — apply-to-job, applicant tracking, employer review with status updates and resume access.
- **Profiles** — job-seeker and employer sub-modules, each with guarded, role-specific endpoints.
- **Auth** — JWKS-verified JWT strategy plus a Supabase webhook that mirrors users locally.
- **Storage** — a swappable object-store abstraction shared by resume and company-logo uploads.

2.2 Design patterns

Repository pattern. Persistence is confined to dedicated `*-repo.service.ts` providers (jobs, users, applications, employer profiles). Controllers and domain services never touch TypeORM directly, so query logic is centralised and the data layer is mockable in tests.

Strategy pattern. File storage sits behind a `StorageService` interface bound by DI tokens. Production resolves the Supabase adapter; tests bind an in-memory fake — the same call sites, a swappable transport. Authentication is likewise a Passport strategy, isolating token verification from route handlers.

2.3 Stateless, JWKS-verified authentication

Bearer tokens minted by Supabase are verified locally: the strategy fetches Supabase's public signing keys over JWKS (cached, rate-limited), validates the RS256 signature, issuer, and expiry, then lazily provisions a mirrored local user on first authenticated request. A database webhook keeps the local mirror in sync — including deletions — guarded by a shared secret header.

```
Bearer JWT -> JWKS RS256 verify (cached) -> claims check -> findOrCreate local user
```

2.4 Cross-cutting hardening

- **Input validation:** global whitelist `ValidationPipe` rejects unknown fields; DTOs enforce shape and types.
- **Rate limiting:** `@nestjs/throttler` applied as a global guard, configurable per environment.
- **Consistent errors:** a global exception filter normalises every error response.
- **File access:** private buckets with short-lived signed URLs; uploads validated by real file-type sniffing, not extensions.

- **Events:** EventEmitter + TypeORM entity subscribers decouple side effects (e.g. cascade cleanup) from request handling.
- **CORS:** configurable CORS allow-list resolved at bootstrap.

2.5 Deployment

A multi-stage Docker build compiles in a full toolchain image and ships only production dependencies and compiled output in the runtime image, keeping the deployed artifact small. The service is stateless, so it scales horizontally behind a load balancer without sticky sessions.

3. Request Lifecycle

A representative authenticated write — an employer publishing a job — passes through the following layers:

#	Layer	Responsibility
1	CORS + Throttler guard	Origin check and per-window rate limiting
2	JWT auth guard / strategy	JWKS signature, issuer & expiry verification
3	Role / profile guard	Ensures caller owns an employer profile
4	ValidationPipe + DTO	Whitelist and type-coerce the request body
5	Controller	Thin HTTP adapter, no business logic
6	Domain service	Business rules & lifecycle transitions
7	Repository service	TypeORM persistence to PostgreSQL
8	Response DTO / mapper	Shape the outgoing payload

4. Technology Stack

Concern	Choice	Why
Framework	NestJS 11 (TypeScript)	Modular DI, guards/pipes/filters as first-class primitives
Database	PostgreSQL + TypeORM	Relational integrity, migrations, entity subscribers
Auth	Supabase + passport-jwt + jwks-rsa	Stateless RS256 verification, no credential handling
Object storage	Supabase Storage (private)	Signed-URL access behind a swappable interface
Rate limiting	@nestjs/throttler	Abuse protection as a global guard
Events	@nestjs/event-emitter	Decouple side effects from request handling
API docs	@nestjs/swagger (OpenAPI)	Live, typed contract at /api
Testing	Jest + Supertest + MSW	End-to-end suites with mocked external services
Packaging	Docker multi-stage + Compose	Small, reproducible production image

5. Measurable Results

The following figures are counted directly from the repository and are reproducible (Appendix A):

- **9 NestJS modules** spanning 5 business domains (auth, profiles, jobs, applications, storage) with clear boundaries.
- **24 REST endpoints** across public browsing, authenticated CRUD, applicant tracking, and employer review flows.

- **7 TypeORM migrations** evolving 4 core entities under version control.
- **14 end-to-end test suites, 296 tests, all passing** exercising real HTTP paths end-to-end (auth, jobs, applications, profiles, webhooks, soft-delete cascade), with Supabase and file-type checks mocked via MSW.
- **96.2% / 75.4% / 95.1% / 96.5% code coverage** measured across the whole application (statements / branches / functions / lines) from a single live run of the full E2E suite against a real PostgreSQL instance — no mocked database.
- **~3,500 req/sec average (3,231 req/sec minimum, p97.5 latency 17 ms, avg latency 13.7 ms)** sustained on GET /jobs (public listing endpoint, 50 concurrent connections, 20 s, autocannon) against a local Dockerised PostgreSQL instance, once warmed up, with 0 failed requests — 70,000+ requests served over the run. Method and caveats in Appendix C.

Caveat: this number was measured running the TypeScript source directly under ts-node (development mode), not the optimised production build (`node dist/main.js` inside the Docker image), and with the request-rate limiter deliberately raised for the test — production defaults to 10 requests/60 s per IP (see §2.4), so this figure reflects the server’s raw request-handling capacity, not its default-configured behaviour under abuse. Single machine, single instance, no concurrent load beyond the benchmark client itself. A run fired immediately after process start measured noticeably lower (~2,400 req/sec, higher latency) — that gap is JIT/connection-pool warm-up, not a second data point on capacity, so it is not the figure quoted above.

Appendix A — Reproducing the coverage numbers

Requires a `jest-e2e.coverage.json` at the repo root (a root-scoped copy of `test/jest-e2e.json` with `collectCoverageFrom` pointed at `src/`). Run from the repository root:

```
docker compose -f docker/test/docker-compose.yml --env-file .env.test up -d --wait
pnpm exec dotenv -e .env.test -- jest --config ./jest-e2e.coverage.json \
  --runInBand --coverage
docker compose -f docker/test/docker-compose.yml --env-file .env.test down -v
rm jest-e2e.coverage.json
```

Appendix B — Full Coverage Report (per file)

Raw output of the command in Appendix A, run against a live PostgreSQL instance in Docker. Directory subtotal rows are shaded; individual files are listed beneath each.

Path	Stmts	Branch	Funcs	Lines	Uncovered lines
All files (project total)	96.22	75.41	95.06	96.52	
src	97.77	75	75	97.43	
app.controller.ts	100	83.33	100	100	33
app.module.ts	100	66.66	100	100	20,34
app.service.ts	80	100	0	66.66	6
src/common/constants	100	100	100	100	
application.ts / config.ts / database.ts / event.ts / job.ts / paging.ts / strategy.ts / swagger.ts	100	100	100	100	
src/common/decorators	85.71	75	100	100	
current-user.decorator.ts	85.71	75	100	100	10
src/common/dto	100	100	100	100	
paging-query.dto.ts	100	100	100	100	
src/common/exception-filters	100	85.71	100	100	
http-exception.filter.ts	100	85.71	100	100	27,32
src/common/guards	96.15	81.81	100	95.45	
jwt-auth.guard.ts	100	100	100	100	
webhook-secret.guard.ts	95	81.81	100	94.44	42
src/common/strategies	100	87.5	100	100	
supabase-jwt.strategy.ts	100	87.5	100	100	16
src/modules/applications	100	75	100	100	
application-review.controller.ts	100	75	100	100	27-94
application.controller.ts	100	75	100	100	28-73
application.module.ts	100	100	100	100	
job-application-review.controller.ts	100	75	100	100	24-52
job-application.controller.ts	100	75	100	100	27-63
src/modules/applications/domain	100	100	100	100	
application-status.ts	100	100	100	100	
src/modules/applications/dto	100	75	100	100	
applicant-profile-response.dto.ts	100	100	100	100	
application-detail-response.dto.ts	100	75	100	100	15
application-list-response.dto.ts	100	100	100	100	
application-response.dto.ts	100	75	100	100	34-42
application-resume-response.dto.ts	100	100	100	100	
application-review-list-response.dto.ts	100	100	100	100	
application-review-response.dto.ts	100	75	100	100	10
create-application.dto.ts	100	100	100	100	
list-job-applications-query.dto.ts	100	75	100	100	15
list-my-applications-query.dto.ts	100	100	100	100	
update-application-status.dto.ts	100	75	100	100	15

Path	Stmts	Branch	Funcs	Lines	Uncovered lines
src/modules/applications/entities	100	77.77	100	100	
application.entity.ts	100	77.77	100	100	44-79
src/modules/applications/mappers	100	100	100	100	
application.mapper.ts	100	100	100	100	
src/modules/applications/services	97.18	89.58	100	97.1	
application-repo.service.ts	97.18	89.58	100	97.1	134,242
src/modules/auth	96.96	75	75	96.29	
auth.controller.ts	95.23	75	75	94.11	31
auth.module.ts	100	100	100	100	
src/modules/auth/webhooks	100	80.76	100	100	
supabase-user-webhook.dto.ts	100	100	100	100	
supabase-users-webhook.controller.ts	100	79.16	100	100	25-49,60-68
src/modules/jobs	100	75.92	100	100	
job.controller.ts	100	75.92	100	100	47,83-221
job.module.ts	100	100	100	100	
src/modules/jobs/domain	100	100	100	100	
job-status.ts	100	100	100	100	
src/modules/jobs/dto	98.57	75	100	98.57	
browse-jobs-query.dto.ts	93.75	75	100	93.75	18
create-job.dto.ts	100	75	100	100	50
job-list-response.dto.ts	100	100	100	100	
job-paging-query.dto.ts	100	100	100	100	
job-response.dto.ts	100	75	100	100	32-56
list-my-jobs-query.dto.ts	100	100	100	100	
public-job-detail-response.dto.ts	100	75	100	100	22
update-job-status.dto.ts	100	75	100	100	18
update-job.dto.ts	100	100	100	100	
src/modules/jobs/entities	100	81.25	100	100	
job.entity.ts	100	81.25	100	100	48-87
src/modules/jobs/services	100	96.29	100	100	
job-repo.service.ts	100	96.29	100	100	45
src/modules/profiles	100	100	100	100	
profile.module.ts	100	100	100	100	
src/modules/profiles/decorators	85.71	62.5	100	100	
current-employer-profile.decorator.ts	85.71	50	100	100	13-14
current-job-seeker-profile.decorator.ts	85.71	75	100	100	12
src/modules/profiles/entities	100	75	100	100	
profile.entity.ts	100	75	100	100	39-119
src/modules/profiles/guards	93.75	75	100	92.85	
employer-profile.guard.ts	93.75	75	100	92.85	46
job-seeker-profile.guard.ts	93.75	75	100	92.85	41
src/modules/profiles/modules/employee-profile	97.61	70.58	100	97.43	
employer-profile-repo.service.ts	95	68.75	100	94.73	51,108

Path	Stmts	Branch	Funcs	Lines	Uncovered lines
employer-profile.controller.ts	100	71.42	100	100	40,66-131
employer-profile.module.ts	100	100	100	100	
src/modules/profiles/modules/employee-profile/domain	85.71	50	100	85.71	
logo-key.ts	85.71	50	100	85.71	24
src/modules/profiles/modules/employee-profile/dto	100	75	100	100	
create-employer-profile.dto.ts	100	100	100	100	
employer-profile-response.dto.ts	100	75	100	100	43-47
public-employer-profile.dto.ts	100	100	100	100	
update-employer-profile.dto.ts	100	100	100	100	
src/modules/profiles/modules/employee-profile/validators	92.3	83.33	100	100	
logo-file-type.validator.ts	92.3	83.33	100	100	27
src/modules/profiles/modules/job-seeker-profile	100	72.54	100	100	
job-seeker-profile.controller.ts	100	72.54	100	100	43,82-149
job-seeker-profile.module.ts	100	100	100	100	
src/modules/profiles/modules/job-seeker-profile/domain	100	100	100	100	
resume-key.ts	100	100	100	100	
src/modules/profiles/modules/job-seeker-profile/dto	100	75	100	100	
create-job-seeker-profile.dto.ts	100	100	100	100	
job-seeker-profile-response.dto.ts	100	75	100	100	32-35
update-job-seeker-profile.dto.ts	100	100	100	100	
src/modules/profiles/modules/job-seeker-profile/services	90.69	63.63	100	90.24	
job-seeker-profile.service.ts	90.69	63.63	100	90.24	60-67,123
src/modules/profiles/modules/job-seeker-profile/validators	88.23	81.81	100	100	
resume-file-type.validator.ts	88.23	81.81	100	100	29-32
src/modules/profiles/subscribers	100	75	100	100	
profile.subscriber.ts	100	75	100	100	19
src/modules/storage	100	100	100	100	
storage.module.ts / storage.tokens.ts	100	100	100	100	
src/modules/storage/services	52.83	25	53.84	50	
in-memory-storage.service.ts	94.73	75	100	94.11	33
supabase-storage.service.ts	29.41	16.66	14.28	25.8	26-97
src/modules/users	100	100	100	100	
user.module.ts	100	100	100	100	
src/modules/users/entities	100	66.66	100	100	
user.entity.ts	100	66.66	100	100	58-74
src/modules/users/services	100	87.5	100	100	
user-repo.service.ts	100	87.5	100	100	19

Weakest spot: src/modules/storage/services/supabase-storage.service.ts sits at 29.41% statements — the production Supabase storage adapter is barely exercised because the E2E suite binds the in-memory storage fake instead (by design, per the Strategy pattern in §2.2). Real Supabase upload/signed-URL/delete behaviour is currently unverified by tests.

Appendix C — Producing a real throughput number

Pitfall 1 — rate limiting: the global ThrottlerGuard (§2.4) defaults to 10 requests per 60 s per IP. Load-testing an instance with that default active measures the rate limiter, not the server — nearly every request comes back 429 and the reported req/sec is meaningless. The limit must be raised for the test.

Pitfall 2 — cold start: starting the server and immediately firing the load client in the same command (`server & ; autocannon ...`) races ts-node's multi-second cold compile. The benchmark's early seconds hit a port nothing is listening on yet, producing connection-refused errors and wildly inflated average latency that has nothing to do with the server's real performance. Wait for the app to report ready before starting the load test.

This report's number was produced against a disposable instance — the test Postgres container, run via ts-node (development mode, not the production build), on a port separate from any real deployment, so the normal dev/prod environment is never touched:

```
docker compose -f docker/test/docker-compose.yml --env-file .env.test up -d --wait
cp .env.test .env.bench
sed -i "s/^THROTTLE_LIMIT=.*\/THROTTLE_LIMIT=1000000\/" .env.bench
PORT=3001 pnpm exec dotenv -e .env.bench -- \
  pnpm exec ts-node -r tsconfig-paths/register src/main.ts &
until curl -s -o /dev/null http://localhost:3001/health; do sleep 1; done
npx autocannon -c 50 -d 20 http://localhost:3001/jobs
```

Run it twice back-to-back and keep the second result: the first run against a freshly started process measured ~2,400 req/sec here; an immediate second run against the now-warm process measured ~3,500 req/sec with tighter latency. The gap is JIT/connection-pool warm-up, and the warmed figure is the one worth quoting as steady-state capacity.

Then clean up — stop the ts-node process, tear down the container, delete the temp env file:

```
docker compose -f docker/test/docker-compose.yml --env-file .env.test down -v
rm .env.bench
```

Only publish numbers you have actually observed, and note how they were produced — this figure reflects raw request-handling capacity under a dev-mode process on one machine, not a tuned production deployment.